Actividad PrÃ¡ctica de la Semana Seis: ImplementaciÃ³n del Modelo FÃ-sico en SQL Server y ConexiÃ³n de la Ventana de Ingreso en Java

Desarrollo de SoluciÃ³n Empresarial con Arquitectura Modelo Vista Controlador y Control de Acceso por Roles

Alessandro Villogas Gaspar

Escuela Profesional de IngenierÃ-a de Sistemas y ComputaciÃ³n

Facultad de IngenierÃ-a, Universidad Peruana Los Andes

Desarrollo de Aplicaciones I

Mg. Ing. RaÃºl FernÃ¡ndez Bejarano

Octubre de 2026

Modelo FÃ-sico de Base de Datos en Microsoft SQL Server

El modelo fÃ-sico constituye la materializaciÃ³n tÃ©cnica del esquema relacional en el motor Microsoft SQL Server para la base de datos BD_NEXORA. Se establecen tipos de datos adecuados, Ã-ndices de bÃºsqueda acelerada, restricciones de verificaciÃ³n y procedimientos almacenados que encapsulan la lÃ³gica transaccional de autenticaciÃ³n e inventario.

Tabla 1

EspecificaciÃ³n TÃ©cnica de Tablas FÃ-sicas y Restricciones de Integridad

Tabla FÃ-sica	DefiniciÃ³n de Columnas y Tipos	Restricciones de Integridad Aplicadas
dbo.rol	id_rol entero autonumÃ©rico, nombre texto de treinta caracteres, descripcion texto, nivel_acceso entero	Clave primaria en id_rol, nombre Ãºnico, verificaciÃ³n de nivel entre uno y cuatro
dbo.usuario	id_usuario entero autonumÃ©rico, username texto de cuarenta caracteres, clave texto, nombre_completo texto, correo texto, id_rol entero, activo booleano, fecha_registro fecha y hora	Clave primaria en id_usuario, cuenta y correo Ãºnicos, clave forÃ¡nea hacia dbo.rol
dbo.sesion_acceso	id_sesion entero autonumÃ©rico, id_usuario entero, fecha_ingreso fecha y hora, ip_cliente texto, exito booleano	Clave primaria en id_sesion, clave forÃ¡nea hacia dbo.usuario, fecha por defecto del sistema
dbo.categoria	id_categoria entero autonumÃ©rico, nombre texto de cincuenta caracteres, descripcion texto, activo booleano	Clave primaria en id_categoria, nombre Ãºnico, estado activo por defecto
dbo.producto	codigo alfanumÃ©rico de diez caracteres, nombre texto, id_categoria entero, precio decimal con dos dÃ-gitos, stock entero, estado texto, fecha_registro fecha y hora	Clave primaria en codigo, clave forÃ¡nea hacia dbo.categoria, precio mayor que cero, existencias no negativas, estado disponible o agotado
dbo.pedido	id_pedido entero autonumÃ©rico, id_usuario entero, codigo_producto texto, cantidad entera, precio_unitario decimal, fecha_pedido fecha y hora	Clave primaria en id_pedido, claves forÃ¡neas hacia usuario y producto, cantidad estrictamente positiva

*Nota.*Restricciones de integridad y tipos de datos del modelo fÃ-sico implementado en Transact SQL.

Script de ImplementaciÃ³n Transact SQL para Microsoft SQL Server

A continuaciÃ³n se presenta el cÃ³digo de definiciÃ³n y manipulaciÃ³n de datos contenido en el repositorio oficial del proyecto para la creaciÃ³n Ã-ntegra de la base de datos BD_NEXORA.

```
/* ============================================================================
 * UNIVERSIDAD PERUANA LOS ANDES - FACULTAD DE INGENIERÍA
 * Curso      : Desarrollo de Aplicaciones I (Semana 05 - Semana 06)
 * Docente    : Mg. Ing. Raúl Fernández Bejarano
 * Estudiante : Alessandro
 * Sistema    : NEXORA - Sistema Web de Control de Inventario y Roles
 * Base Datos : BD_NEXORA (Microsoft SQL Server / Azure SQL Database)
 * Descripción: Script del Modelo Físico (DDL + DML + Roles + Inicio de Sesión)
 * ============================================================================
*/

-- 1. CREACIÓN DE LA BASE DE DATOS BD_NEXORA
IF DB_ID(N'BD_NEXORA') IS NULL
BEGIN
    CREATE DATABASE BD_NEXORA;
END
GO

USE BD_NEXORA;
GO

-- ============================================================================
-- 2. ELIMINACIÓN PREVIA DE OBJETOS (SI EXISTEN) PARA RE-EJECUCIÓN LIMPIA
-- ============================================================================
IF OBJECT_ID(N'dbo.sp_validar_usuario', N'P') IS NOT NULL DROP PROCEDURE
dbo.sp_validar_usuario;
IF OBJECT_ID(N'dbo.sp_listar_productos', N'P') IS NOT NULL DROP PROCEDURE
dbo.sp_listar_productos;
IF OBJECT_ID(N'dbo.sp_registrar_producto', N'P') IS NOT NULL DROP PROCEDURE
dbo.sp_registrar_producto;
IF OBJECT_ID(N'dbo.sp_actualizar_producto', N'P') IS NOT NULL DROP PROCEDURE
dbo.sp_actualizar_producto;
IF OBJECT_ID(N'dbo.sp_eliminar_producto', N'P') IS NOT NULL DROP PROCEDURE
dbo.sp_eliminar_producto;
IF OBJECT_ID(N'dbo.sp_registrar_pedido', N'P') IS NOT NULL DROP PROCEDURE
dbo.sp_registrar_pedido;
IF OBJECT_ID(N'dbo.vw_productos_detalle', N'V') IS NOT NULL DROP VIEW
dbo.vw_productos_detalle;
IF OBJECT_ID(N'dbo.vw_usuarios_roles', N'V') IS NOT NULL DROP VIEW
dbo.vw_usuarios_roles;
IF OBJECT_ID(N'dbo.pedido', N'U') IS NOT NULL DROP TABLE dbo.pedido;
IF OBJECT_ID(N'dbo.producto', N'U') IS NOT NULL DROP TABLE dbo.producto;
IF OBJECT_ID(N'dbo.categoria', N'U') IS NOT NULL DROP TABLE dbo.categoria;
IF OBJECT_ID(N'dbo.sesion_acceso', N'U') IS NOT NULL DROP TABLE
dbo.sesion_acceso;
IF OBJECT_ID(N'dbo.usuario', N'U') IS NOT NULL DROP TABLE dbo.usuario;
IF OBJECT_ID(N'dbo.rol', N'U') IS NOT NULL DROP TABLE dbo.rol;
GO

-- ============================================================================
-- 3. MODELO FÍSICO DE TABLAS (DDL - 3ra Forma Normal)
-- ============================================================================
```

```
-- 3.1 Tabla ROL: Perfiles y niveles de permiso en el sistema NEXORA
CREATE TABLE dbo.rol (
    id_rol       INT IDENTITY(1,1) NOT NULL,
    nombre       NVARCHAR(30)      NOT NULL,
    descripcion  NVARCHAR(150)     NOT NULL,
    nivel_acceso INT               NOT NULL CONSTRAINT DF_rol_nivel DEFAULT (4),
    CONSTRAINT PK_rol PRIMARY KEY CLUSTERED (id_rol),
    CONSTRAINT UQ_rol_nombre UNIQUE (nombre),
    CONSTRAINT CK_rol_nivel CHECK (nivel_acceso IN (1, 2, 3, 4))
);
GO

-- 3.2 Tabla USUARIO: Credenciales de la Ventana de Ingreso vinculadas a un Rol
CREATE TABLE dbo.usuario (
    id_usuario      INT IDENTITY(1,1) NOT NULL,
    username        NVARCHAR(40)      NOT NULL,
    clave           NVARCHAR(120)     NOT NULL,
    nombre_completo NVARCHAR(100)     NOT NULL,
    correo          NVARCHAR(100)     NOT NULL,
    id_rol          INT               NOT NULL,
    activo          BIT               NOT NULL CONSTRAINT DF_usuario_activo
DEFAULT (1),
    fecha_registro  DATETIME2(0)      NOT NULL CONSTRAINT DF_usuario_fecha
DEFAULT (SYSDATETIME()),
    CONSTRAINT PK_usuario PRIMARY KEY CLUSTERED (id_usuario),
    CONSTRAINT UQ_usuario_username UNIQUE (username),
    CONSTRAINT UQ_usuario_correo UNIQUE (correo),
    CONSTRAINT FK_usuario_rol FOREIGN KEY (id_rol) REFERENCES dbo.rol (id_rol)
);
GO

-- 3.3 Tabla SESION_ACCESO: Bitácora de auditoría de inicios de sesión (Login)
CREATE TABLE dbo.sesion_acceso (
    id_sesion     INT IDENTITY(1,1) NOT NULL,
    id_usuario    INT               NOT NULL,
    fecha_ingreso DATETIME2(0)      NOT NULL CONSTRAINT DF_sesion_fecha DEFAULT
(SYSDATETIME()),
    ip_cliente    VARCHAR(45)       NOT NULL CONSTRAINT DF_sesion_ip DEFAULT
('127.0.0.1'),
    exito         BIT               NOT NULL CONSTRAINT DF_sesion_exito DEFAULT
(1),
    CONSTRAINT PK_sesion_acceso PRIMARY KEY CLUSTERED (id_sesion),
    CONSTRAINT FK_sesion_usuario FOREIGN KEY (id_usuario) REFERENCES dbo.usuario
(id_usuario)
);
GO

-- 3.4 Tabla CATEGORIA: Clasificación del catálogo de productos NEXORA
CREATE TABLE dbo.categoria (
    id_categoria INT IDENTITY(1,1) NOT NULL,
    nombre       NVARCHAR(50)      NOT NULL,
    descripcion  NVARCHAR(150)     NULL,
```

```
    activo       BIT               NOT NULL CONSTRAINT DF_categoria_activo
DEFAULT (1),
    CONSTRAINT PK_categoria PRIMARY KEY CLUSTERED (id_categoria),
    CONSTRAINT UQ_categoria_nombre UNIQUE (nombre)
);
GO

-- 3.5 Tabla PRODUCTO: Entidad principal del inventario (Producto.java)
CREATE TABLE dbo.producto (
    codigo         VARCHAR(10)    NOT NULL,
    nombre         NVARCHAR(120)  NOT NULL,
    id_categoria   INT            NOT NULL,
    precio         DECIMAL(10,2)  NOT NULL,
    stock          INT            NOT NULL,
    estado         NVARCHAR(20)   NOT NULL,
    fecha_registro DATETIME2(0)   NOT NULL CONSTRAINT DF_producto_fecha DEFAULT
(SYSDATETIME()),
    CONSTRAINT PK_producto PRIMARY KEY CLUSTERED (codigo),
    CONSTRAINT FK_producto_categoria FOREIGN KEY (id_categoria) REFERENCES
dbo.categoria (id_categoria),
    CONSTRAINT CK_producto_precio CHECK (precio > 0),
    CONSTRAINT CK_producto_stock CHECK (stock >= 0),
    CONSTRAINT CK_producto_estado CHECK (estado IN (N'Disponible', N'Agotado'))
);
GO

-- 3.6 Tabla PEDIDO: Solicitudes y compras realizadas por usuarios con rol
Cliente
CREATE TABLE dbo.pedido (
    id_pedido       INT IDENTITY(1,1) NOT NULL,
    id_usuario      INT               NOT NULL,
    codigo_producto VARCHAR(10)       NOT NULL,
    cantidad        INT               NOT NULL,
    precio_unitario DECIMAL(10,2)     NOT NULL,
    fecha_pedido    DATETIME2(0)      NOT NULL CONSTRAINT DF_pedido_fecha DEFAULT
(SYSDATETIME()),
    CONSTRAINT PK_pedido PRIMARY KEY CLUSTERED (id_pedido),
    CONSTRAINT FK_pedido_usuario FOREIGN KEY (id_usuario) REFERENCES dbo.usuario
(id_usuario),
    CONSTRAINT FK_pedido_producto FOREIGN KEY (codigo_producto) REFERENCES
dbo.producto (codigo),
    CONSTRAINT CK_pedido_cantidad CHECK (cantidad > 0),
    CONSTRAINT CK_pedido_precio CHECK (precio_unitario > 0)
);
GO

CREATE NONCLUSTERED INDEX IX_producto_nombre ON dbo.producto (nombre);
CREATE NONCLUSTERED INDEX IX_producto_id_categoria ON dbo.producto
(id_categoria);
CREATE NONCLUSTERED INDEX IX_usuario_id_rol ON dbo.usuario (id_rol);
CREATE NONCLUSTERED INDEX IX_pedido_id_usuario ON dbo.pedido (id_usuario);
GO
```

```
-- ============================================================================
-- 4. CARGA DE DATOS INICIALES (DML - Roles, Usuarios, Productos y Pedidos)
-- ============================================================================

-- 4.1 Los 4 Roles del sistema NEXORA (incluyendo Cliente)
INSERT INTO dbo.rol (nombre, descripcion, nivel_acceso) VALUES
(N'Administrador', N'Control total: registrar, editar, eliminar productos y
consultar reportes',          1),
(N'Almacén',       N'Operación de inventario: registrar, editar productos y
consultar reportes',          2),
(N'Cliente',       N'Catálogo de cliente: explorar productos, realizar
compras/pedidos y ver mis pedidos', 3),
(N'Consulta',      N'Solo lectura: listar, buscar productos y visualizar
reportes',                        4);
GO

-- 4.2 Usuarios para la Ventana de Ingreso (UsuarioService.java)
INSERT INTO dbo.usuario (username, clave, nombre_completo, correo, id_rol,
activo) VALUES
(N'admin',    N'admin123',   N'Alessandro · Administrador General',
N'admin@nexora.pe',    1, 1),
(N'almacen',  N'almacen123', N'Carlos Vega · Operador de Almacén',
N'almacen@nexora.pe',  2, 1),
(N'cliente',  N'cliente123', N'Valeria Mendoza · Cliente Corporativo',
N'cliente@nexora.pe',  3, 1),
(N'invitado', N'invitado',   N'Lucía Torres · Analista Consulta',
N'consulta@nexora.pe', 4, 1);
GO

-- 4.3 Registro inicial en bitácora de inicios de sesión
INSERT INTO dbo.sesion_acceso (id_usuario, ip_cliente, exito) VALUES
(1, '127.0.0.1', 1),
(2, '127.0.0.1', 1),
(3, '127.0.0.1', 1);
GO

-- 4.4 Categorías oficiales del sistema NEXORA
INSERT INTO dbo.categoria (nombre, descripcion) VALUES
(N'Computadoras',   N'Laptops, equipos de escritorio y estaciones de trabajo'),
(N'Accesorios',     N'Periféricos: mouse, teclados, audífonos y webcams'),
(N'Monitores',      N'Pantallas LED, IPS y monitores profesionales'),
(N'Impresoras',     N'Impresoras multifuncionales y de tinta continua'),
(N'Almacenamiento', N'Unidades de estado sólido SSD, discos duros y memorias
USB'),
(N'Redes',          N'Routers inalámbricos, switches y equipos de conectividad');
GO

-- 4.5 Los 12 productos iniciales del inventario NEXORA
INSERT INTO dbo.producto (codigo, nombre, id_categoria, precio, stock, estado)
VALUES
('P001', N'Laptop Lenovo IdeaPad 3',   1, 2500.00, 10, N'Disponible'),
('P002', N'Mouse Logitech M170',       2,   80.00, 25, N'Disponible'),
('P003', N'Teclado Redragon Kumara',   2,  150.00,  0, N'Agotado'),
```

```
('P004', N'Monitor LG 24" IPS',        3,  650.00,  8, N'Disponible'),
('P005', N'Impresora Epson L3250',     4,  799.00,  4, N'Disponible'),
('P006', N'SSD Kingston A400 480GB',   5,  165.00, 30, N'Disponible'),
('P007', N'USB SanDisk Ultra 64GB',    5,   35.00, 60, N'Disponible'),
('P008', N'Audífonos HyperX Cloud II', 2,  280.00,  0, N'Agotado'),
('P009', N'Router TP-Link Archer C6',  6,  189.00, 12, N'Disponible'),
('P010', N'Laptop HP 15 Ryzen 5',      1, 2890.00,  5, N'Disponible'),
('P011', N'Webcam Logitech C920',      2,  320.00,  7, N'Disponible'),
('P012', N'Switch TP-Link 8 puertos',  6,   95.00,  0, N'Agotado');
GO

-- 4.6 Pedidos iniciales del rol Cliente (dbo.pedido)
INSERT INTO dbo.pedido (id_usuario, codigo_producto, cantidad, precio_unitario)
VALUES
(3, 'P002', 2,  80.00),
(3, 'P006', 1, 165.00);
GO

-- ============================================================================
-- 5. VISTAS Y PROCEDIMIENTOS ALMACENADOS (Login + Roles + CRUD Productos)
-- ============================================================================

-- 5.1 Vista de usuarios con su rol y nivel de acceso
CREATE VIEW dbo.vw_usuarios_roles
AS
SELECT
    u.id_usuario,
    u.username,
    u.nombre_completo,
    u.correo,
    r.id_rol,
    r.nombre AS rol,
    r.nivel_acceso,
    r.descripcion AS permisos,
    u.activo,
    u.fecha_registro
FROM dbo.usuario u
INNER JOIN dbo.rol r ON u.id_rol = r.id_rol;
GO

-- 5.2 Vista detallada de productos con categoría y valor de inventario
CREATE VIEW dbo.vw_productos_detalle
AS
SELECT
    p.codigo,
    p.nombre,
    c.nombre AS categoria,
    p.precio,
    p.stock,
    p.estado,
    CAST(p.precio * p.stock AS DECIMAL(12,2)) AS valor_inventario,
    p.fecha_registro
FROM dbo.producto p
```

```
INNER JOIN dbo.categoria c ON p.id_categoria = c.id_categoria;
GO

-- 5.3 Procedimiento almacenado para la Ventana de Ingreso (Login con Rol y
Bitácora)
CREATE PROCEDURE dbo.sp_validar_usuario
    @username   NVARCHAR(40),
    @clave      NVARCHAR(120),
    @ip_cliente VARCHAR(45) = '127.0.0.1'
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @id_usr INT;

    SELECT @id_usr = u.id_usuario
    FROM dbo.usuario u
    WHERE LOWER(u.username) = LOWER(LTRIM(RTRIM(@username)))
      AND u.clave = @clave
      AND u.activo = 1;

    IF @id_usr IS NOT NULL
    BEGIN
        INSERT INTO dbo.sesion_acceso (id_usuario, ip_cliente, exito)
        VALUES (@id_usr, @ip_cliente, 1);

        SELECT id_usuario, username, nombre_completo, correo, rol, nivel_acceso,
permisos, activo
        FROM dbo.vw_usuarios_roles
        WHERE id_usuario = @id_usr;
    END
END
GO

-- 5.4 Procedimiento para listar todos los productos
CREATE PROCEDURE dbo.sp_listar_productos
AS
BEGIN
    SET NOCOUNT ON;
    SELECT codigo, nombre, categoria, precio, stock, estado
    FROM dbo.vw_productos_detalle
    ORDER BY codigo ASC;
END
GO

-- 5.5 Procedimiento para registrar un nuevo producto
CREATE PROCEDURE dbo.sp_registrar_producto
    @codigo    VARCHAR(10),
    @nombre    NVARCHAR(120),
    @categoria NVARCHAR(50),
    @precio    DECIMAL(10,2),
    @stock     INT
AS
```

```
BEGIN
    SET NOCOUNT ON;
    DECLARE @id_cat INT;
    SELECT @id_cat = id_categoria FROM dbo.categoria WHERE LOWER(nombre) =
LOWER(@categoria);

    IF @id_cat IS NULL
        THROW 50001, 'La categoría especificada no existe en BD_NEXORA.', 1;

    DECLARE @estado NVARCHAR(20) = CASE WHEN @stock > 0 THEN N'Disponible' ELSE
N'Agotado' END;

    INSERT INTO dbo.producto (codigo, nombre, id_categoria, precio, stock,
estado)
    VALUES (UPPER(LTRIM(RTRIM(@codigo))), LTRIM(RTRIM(@nombre)), @id_cat,
@precio, @stock, @estado);
END
GO

-- 5.6 Procedimiento para actualizar un producto existente
CREATE PROCEDURE dbo.sp_actualizar_producto
    @codigo    VARCHAR(10),
    @nombre    NVARCHAR(120),
    @categoria NVARCHAR(50),
    @precio    DECIMAL(10,2),
    @stock     INT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @id_cat INT;
    SELECT @id_cat = id_categoria FROM dbo.categoria WHERE LOWER(nombre) =
LOWER(@categoria);

    IF @id_cat IS NULL
        THROW 50001, 'La categoría especificada no existe en BD_NEXORA.', 1;

    DECLARE @estado NVARCHAR(20) = CASE WHEN @stock > 0 THEN N'Disponible' ELSE
N'Agotado' END;

    UPDATE dbo.producto
    SET nombre       = LTRIM(RTRIM(@nombre)),
        id_categoria = @id_cat,
        precio       = @precio,
        stock        = @stock,
        estado       = @estado
    WHERE codigo = UPPER(LTRIM(RTRIM(@codigo)));
END
GO

-- 5.7 Procedimiento para eliminar un producto por código
CREATE PROCEDURE dbo.sp_eliminar_producto
    @codigo VARCHAR(10)
```

```
AS
BEGIN
    SET NOCOUNT ON;
    DELETE FROM dbo.producto
    WHERE codigo = UPPER(LTRIM(RTRIM(@codigo)));
END
GO

-- 5.8 Procedimiento para registrar un pedido del rol Cliente y descontar stock
CREATE PROCEDURE dbo.sp_registrar_pedido
    @id_usuario      INT,
    @codigo_producto VARCHAR(10),
    @cantidad        INT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @stock_actual INT, @precio DECIMAL(10,2);

    SELECT @stock_actual = stock, @precio = precio
    FROM dbo.producto
    WHERE codigo = UPPER(LTRIM(RTRIM(@codigo_producto)));

    IF @stock_actual IS NULL
        THROW 50002, 'El producto solicitado no existe en BD_NEXORA.', 1;

    IF @stock_actual < @cantidad
        THROW 50003, 'Stock insuficiente para atender el pedido del cliente.', 1;

    DECLARE @nuevo_stock INT = @stock_actual - @cantidad;
    DECLARE @nuevo_estado NVARCHAR(20) = CASE WHEN @nuevo_stock > 0 THEN
N'Disponible' ELSE N'Agotado' END;

    UPDATE dbo.producto
    SET stock  = @nuevo_stock,
        estado = @nuevo_estado
    WHERE codigo = UPPER(LTRIM(RTRIM(@codigo_producto)));

    INSERT INTO dbo.pedido (id_usuario, codigo_producto, cantidad,
precio_unitario)
    VALUES (@id_usuario, UPPER(LTRIM(RTRIM(@codigo_producto))), @cantidad,
@precio);
END
GO

-- ============================================================================
-- 6. CONSULTAS DE VERIFICACIÓN DEL SISTEMA NEXORA
-- ============================================================================
SELECT * FROM dbo.rol;
SELECT * FROM dbo.vw_usuarios_roles;
SELECT * FROM dbo.sesion_acceso;
SELECT * FROM dbo.categoria;
SELECT * FROM dbo.vw_productos_detalle ORDER BY codigo;
```

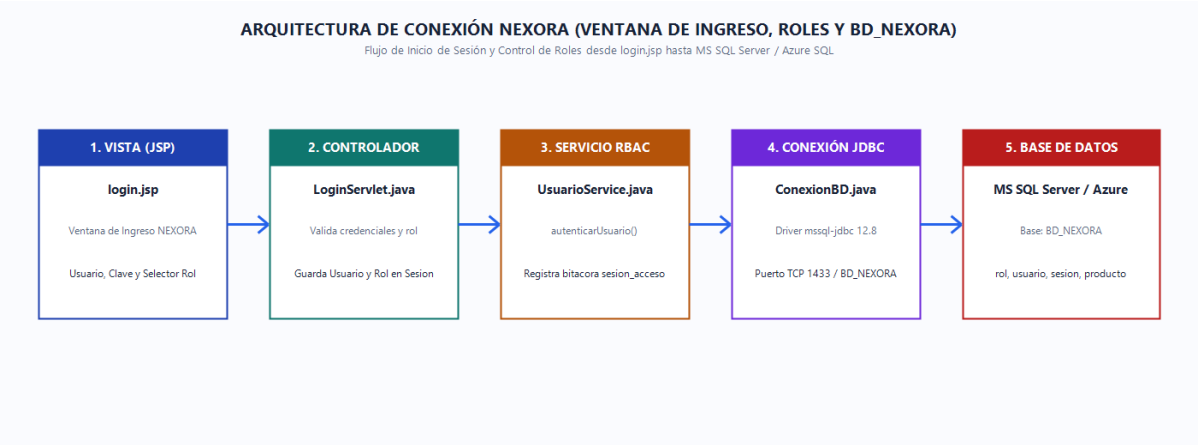
```
SELECT * FROM dbo.pedido;
GO
```

ConexiÃ³n de la AplicaciÃ³n Web en Java con la Base de Datos

La conexiÃ³n entre la interfaz web y Microsoft SQL Server se fundamenta en el patrÃ³n Modelo Vista Controlador. La ventana de ingreso permite al usuario ingresar sus credenciales y seleccionar su perfil de acceso. El controlador valida dicha informaciÃ³n mediante consultas parametrizadas, previene ataques de inyecciÃ³n de cÃ³digo y establece una sesiÃ³n segura mediante identificadores renovados.

Figura 1

Arquitectura de ConexiÃ³n y Flujo de AutenticaciÃ³n de la Ventana de Ingreso



*Nota.*Diagrama multicapa desde la vista JSP hasta el motor relacional SQL Server.

Clase de ConexiÃ³n mediante el Controlador Oficial de Microsoft

El archivo ConexionBD.java centraliza los parÃ¡metros de enlace hacia SQL Server local o Azure SQL Database, implementando un tiempo lÃmite de conexiÃ³n de dos segundos para asegurar alta disponibilidad mediante respaldo en memoria si el motor no se encontrara disponible.

```
package pe.edu.sistema.util;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * Clase de conexión JDBC del Sistema NEXORA para Microsoft SQL Server y Azure
SQL Database (Semana 06).
 *
 * Soporta dos modos de configuración:
 * 1. SQL Server Local (por defecto):
```

```
 *
jdbc:sqlserver://localhost:1433;databaseName=BD_NEXORA;encrypt=true;trustServerCe
rtificate=true;loginTimeout=2;
 *
 * 2. Microsoft Azure SQL Database (mediante variables de entorno o propiedades):
 *    jdbc:sqlserver://srv-
nexora.database.windows.net:1433;database=BD_NEXORA;encrypt=true;trustServerCerti
ficate=false;hostNameInCertificate=*.database.windows.net;loginTimeout=5;
 *
 * Si el motor SQL Server no se encuentra activo en el equipo, el sistema NEXORA
activa
 * automáticamente el modo de respaldo en memoria (ArrayList) sin interrumpir la
aplicación.
 */
public final class ConexionBD {

    private static final String DRIVER =
"com.microsoft.sqlserver.jdbc.SQLServerDriver";

    // Parámetros por defecto para MS SQL Server local (Base de datos BD_NEXORA)
    private static final String DEFAULT_URL =
            "jdbc:sqlserver://localhost:1433;databaseName=BD_NEXORA;"
            + "encrypt=true;trustServerCertificate=true;loginTimeout=2;";
    private static final String DEFAULT_USER = "sa";
    private static final String DEFAULT_PASSWORD = "TuPasswordSQL123";

    private static volatile Boolean bdDisponible = null;
    private static volatile String detalleEstado = "Pendiente de verificación";

    static {
        try {
            Class.forName(DRIVER);
        } catch (ClassNotFoundException e) {
            bdDisponible = Boolean.FALSE;
            detalleEstado = "Driver JDBC de SQL Server no encontrado en el
classpath";
        }
    }

    private ConexionBD() {
    }

    /**
     * Obtiene una nueva conexión JDBC a MS SQL Server o Azure SQL Database.
     *
     * @return instancia de {@link Connection} abierta
     * @throws SQLException si no es posible establecer la conexión
     */
    public static Connection obtenerConexion() throws SQLException {
        String url = leerConfig("DB_URL", DEFAULT_URL);
        String usuario = leerConfig("DB_USER", DEFAULT_USER);
        String clave = leerConfig("DB_PASSWORD", DEFAULT_PASSWORD);
        return DriverManager.getConnection(url, usuario, clave);
```

```
    }

    /**
     * Verifica si la base de datos SQL Server está activa y accesible.
     * El resultado se guarda en caché para no consumir recursos en cada
petición.
     */
    public static boolean estaDisponible() {
        Boolean actual = bdDisponible;
        if (actual != null) {
            return actual;
        }
        return probarConexion();
    }

    /**
     * Fuerza una nueva prueba de conexión contra SQL Server / Azure SQL.
     */
    public static synchronized boolean probarConexion() {
        try (Connection cn = obtenerConexion()) {
            bdDisponible = (cn != null && !cn.isClosed());
            detalleEstado = "Conectado a MS SQL Server (BD_NEXORA)";
            return bdDisponible;
        } catch (SQLException ex) {
            bdDisponible = Boolean.FALSE;
            detalleEstado = "Respaldo en memoria activo (BD_NEXORA sin conexión
local)";
            return false;
        }
    }

    /**
     * Devuelve una descripción legible del estado actual de la persistencia.
     */
    public static String getEstadoTexto() {
        estaDisponible();
        return detalleEstado;
    }

    /**
     * Devuelve la cadena JDBC configurada (sin exponer contraseñas).
     */
    public static String getUrlConfigurada() {
        return leerConfig("DB_URL", DEFAULT_URL);
    }

    private static String leerConfig(String clave, String valorPorDefecto) {
        String prop = System.getProperty(clave);
        if (prop != null && !prop.isBlank()) {
            return prop.trim();
        }
        String env = System.getenv(clave);
```

```
        if (env != null && !env.isBlank()) {
            return env.trim();
        }
        return valorPorDefecto;
    }
}
```

Servicio de AutenticaciÃ³n y Control de Acceso por Roles

El componente UsuarioService.java ejecuta la autenticaciÃ³n mediante sentencias preparadas, registra la bitÃ¡cora de acceso con la direcciÃ³n de red del cliente y permite el auto registro de nuevos clientes en la plataforma.

```
package pe.edu.sistema.service;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import pe.edu.sistema.model.Usuario;
import pe.edu.sistema.util.ConexionBD;

/**
 * Servicio de acceso, registro de clientes y control de roles para la Ventana de
Ingreso
 * del sistema NEXORA (Semana 06).
 * Consulta las tablas dbo.usuario y dbo.rol en Microsoft SQL Server / Azure SQL
(BD_NEXORA).
 * Si el servidor de base de datos no está activo, utiliza el respaldo en
memoria.
 */
public class UsuarioService {

    public static final List<String> ROLES = List.of(
            Usuario.ROL_ADMIN, Usuario.ROL_ALMACEN, Usuario.ROL_CLIENTE,
Usuario.ROL_CONSULTA);

    /** Respaldo en memoria mutable: username -> {id, clave, nombreCompleto,
correo, rol} */
    private static final Map<String, String[]> USUARIOS_MEMORIA = new
LinkedHashMap<>();

    static {
        USUARIOS_MEMORIA.put("admin",    new String[]{"1", "admin123",
"Alessandro · Administrador General", "admin@nexora.pe",    Usuario.ROL_ADMIN});
        USUARIOS_MEMORIA.put("almacen",  new String[]{"2", "almacen123", "Carlos
Vega · Operador de Almacén",  "almacen@nexora.pe",  Usuario.ROL_ALMACEN});
```

```
        USUARIOS_MEMORIA.put("cliente",  new String[]{"3", "cliente123", "Andrea
Paredes · Cliente NEXORA",    "cliente@nexora.pe",  Usuario.ROL_CLIENTE});
        USUARIOS_MEMORIA.put("invitado", new String[]{"4", "invitado",   "Lucía
Torres · Analista Consulta",   "consulta@nexora.pe", Usuario.ROL_CONSULTA});
    }

    private static final String SQL_AUTENTICAR =
            "SELECT u.id_usuario, u.username, u.nombre_completo, u.correo,
r.nombre AS rol, u.activo "
            + "FROM dbo.usuario u "
            + "INNER JOIN dbo.rol r ON u.id_rol = r.id_rol "
            + "WHERE LOWER(u.username) = ? AND u.clave = ? AND u.activo = 1";

    private static final String SQL_REGISTRAR_CLIENTE =
            "INSERT INTO dbo.usuario (username, clave, nombre_completo, correo,
id_rol, activo) "
            + "VALUES (?, ?, ?, ?, (SELECT id_rol FROM dbo.rol WHERE
LOWER(nombre) = LOWER('Cliente')), 1)";

    private static final String SQL_REGISTRAR_SESION =
            "INSERT INTO dbo.sesion_acceso (id_usuario, ip_cliente, exito) VALUES
(?, ?, 1)";

    /**
     * Valida las credenciales y el rol en la Ventana de Ingreso de NEXORA.
     */
    public Usuario autenticarUsuario(String usuario, String clave, String
rolRequerido, String ipCliente) {
        if (usuario == null || clave == null) {
            return null;
        }
        String userNorm = usuario.trim().toLowerCase();
        Usuario encontrado = null;

        // 1. Intentar autenticación contra MS SQL Server / Azure SQL Database
(BD_NEXORA)
        if (ConexionBD.estaDisponible()) {
            try (Connection cn = ConexionBD.obtenerConexion();
                 PreparedStatement ps = cn.prepareStatement(SQL_AUTENTICAR)) {
                ps.setString(1, userNorm);
                ps.setString(2, clave);
                try (ResultSet rs = ps.executeQuery()) {
                    if (rs.next()) {
                        encontrado = new Usuario(
                                rs.getInt("id_usuario"),
                                rs.getString("username"),
                                rs.getString("nombre_completo"),
                                rs.getString("correo"),
                                rs.getString("rol"),
                                rs.getBoolean("activo"));
                    }
                }
                if (encontrado != null) {
```

```
                    registrarBitacoraSesion(cn, encontrado.getIdUsuario(),
ipCliente);
                }
            } catch (SQLException ex) {
                ConexionBD.probarConexion();
            }
        }

        // 2. Respaldo en memoria si SQL Server no está activo
        if (encontrado == null && !ConexionBD.estaDisponible()) {
            String[] datos = USUARIOS_MEMORIA.get(userNorm);
            if (datos != null && datos[1].equals(clave)) {
                encontrado = new Usuario(
                        Integer.parseInt(datos[0]),
                        userNorm,
                        datos[2],
                        datos[3],
                        datos[4],
                        true);
            }
        }

        // 3. Verificar coincidencia de rol si el usuario seleccionó uno
específico en la Ventana de Ingreso
        if (encontrado != null && rolRequerido != null && !rolRequerido.isBlank()
                && !"todos".equalsIgnoreCase(rolRequerido)) {
            if (!encontrado.getRol().equalsIgnoreCase(rolRequerido.trim())) {
                return null;
            }
        }

        return encontrado;
    }

    /**
     * Registra una nueva cuenta con el rol Cliente desde la Ventana de Ingreso.
     *
     * @return mensaje de error si los datos son inválidos o el usuario ya
existe; cadena vacía si fue exitoso
     */
    public synchronized String registrarCliente(String nombreCompleto, String
username, String clave) {
        return registrarCliente(username, clave, nombreCompleto, null);
    }

    public synchronized String registrarCliente(String username, String clave,
String nombreCompleto, String correo) {
        String u = username == null ? "" : username.trim().toLowerCase();
        String c = clave == null ? "" : clave.trim();
        String n = nombreCompleto == null ? "" : nombreCompleto.trim();
        String m = correo == null || correo.isBlank() ? (u +
"@cliente.nexora.pe") : correo.trim().toLowerCase();
```

```
        if (u.length() < 3) {
            return "El usuario del cliente debe tener al menos 3 caracteres.";
        }
        if (c.length() < 4) {
            return "La contraseña debe tener al menos 4 caracteres.";
        }
        if (n.isEmpty()) {
            return "Ingrese el nombre completo del cliente.";
        }
        if (USUARIOS_MEMORIA.containsKey(u)) {
            return "El nombre de usuario '" + u + "' ya está registrado.";
        }

        if (ConexionBD.estaDisponible()) {
            try (Connection cn = ConexionBD.obtenerConexion();
                 PreparedStatement ps =
cn.prepareStatement(SQL_REGISTRAR_CLIENTE)) {
                ps.setString(1, u);
                ps.setString(2, c);
                ps.setString(3, n);
                ps.setString(4, m);
                ps.executeUpdate();
            } catch (SQLException ex) {
                return "No se pudo registrar el cliente en BD_NEXORA (verifique
que el usuario o correo no esté duplicado).";
            }
        }

        String nuevoId = String.valueOf(USUARIOS_MEMORIA.size() + 1);
        USUARIOS_MEMORIA.put(u, new String[]{nuevoId, c, n + " · Cliente NEXORA",
m, Usuario.ROL_CLIENTE});
        return "";
    }

    /** Devuelve el rol si las credenciales son correctas; si no, null. */
    public String autenticar(String usuario, String clave) {
        Usuario u = autenticarUsuario(usuario, clave, null, "127.0.0.1");
        return u != null ? u.getRol() : null;
    }

    /** Lista los perfiles de usuario registrados para visualización del sistema.
*/
    public synchronized List<Usuario> listarUsuariosDemo() {
        List<Usuario> lista = new ArrayList<>();
        for (Map.Entry<String, String[]> e : USUARIOS_MEMORIA.entrySet()) {
            String[] d = e.getValue();
            lista.add(new Usuario(Integer.parseInt(d[0]), e.getKey(), d[2], d[3],
d[4], true));
        }
        return lista;
    }
```

```
    private void registrarBitacoraSesion(Connection cn, int idUsuario, String ip)
{
        try (PreparedStatement ps = cn.prepareStatement(SQL_REGISTRAR_SESION)) {
            ps.setInt(1, idUsuario);
            ps.setString(2, ip != null ? ip : "127.0.0.1");
            ps.executeUpdate();
        } catch (SQLException ignored) {
        }
    }
}
```

Controlador de la Ventana de Ingreso y Cierre de SesiÃ³n

El servlet LoginServlet.java procesa las peticiones de ingreso, destruye sesiones anteriores para mitigar la fijaciÃ³n de sesiÃ³n y crea una sesiÃ³n autenticada con los privilegios correspondientes.

```
package pe.edu.sistema.controller;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;
import java.io.IOException;
import pe.edu.sistema.model.Usuario;
import pe.edu.sistema.service.UsuarioService;
import pe.edu.sistema.util.ConexionBD;

/**
 * Controlador de inicio y cierre de sesión del sistema NEXORA (Ventana de
Ingreso · Semana 06).
 */
@WebServlet(name = "LoginServlet", urlPatterns = {"/login", "/logout"})
public class LoginServlet extends HttpServlet {

    private final UsuarioService usuarios = new UsuarioService();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        if ("/logout".equals(req.getServletPath())) {
            HttpSession sesion = req.getSession(false);
            if (sesion != null) {
                sesion.invalidate();
            }
            resp.sendRedirect(req.getContextPath() + "/login");
            return;
        }
```

```
        HttpSession sesion = req.getSession(false);
        if (sesion != null && sesion.getAttribute("usuario") != null) {
            resp.sendRedirect(req.getContextPath() + "/inicio");
            return;
        }
        if ("1".equals(req.getParameter("probarBD"))) {
            ConexionBD.probarConexion();
        }
        req.setAttribute("roles", UsuarioService.ROLES);
        req.setAttribute("bdConectada", ConexionBD.estaDisponible());
        req.setAttribute("bdEstado", ConexionBD.getEstadoTexto());
        Controlador.vista(req, resp, "login");
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        String accion = req.getParameter("accion");
        if ("registrarCliente".equals(accion)) {
            registrarNuevoCliente(req, resp);
            return;
        }

        String usuario = req.getParameter("usuario");
        String clave = req.getParameter("clave");

        Usuario u = usuarios.autenticarUsuario(usuario, clave, null,
req.getRemoteAddr());

        if (u == null) {
            req.setAttribute("error", "Usuario o contraseña incorrectos.");
            req.setAttribute("usuarioIngresado", usuario);
            req.setAttribute("bdConectada", ConexionBD.estaDisponible());
            req.setAttribute("bdEstado", ConexionBD.getEstadoTexto());
            Controlador.vista(req, resp, "login");
            return;
        }

        String saludo;
        if (u.isCliente()) {
            saludo = "¡Hola, " + u.getNombreCompleto() + "! Bienvenido a la
tienda NEXORA.";
        } else if (u.isAdministrador()) {
            saludo = "Bienvenido al panel de administración general, " +
u.getNombreCompleto() + ".";
        } else if (u.isAlmacen()) {
            saludo = "Bienvenido a la gestión de inventario y almacén, " +
u.getNombreCompleto() + ".";
        } else {
            saludo = "Bienvenido a NEXORA, " + u.getNombreCompleto() + " · Modo
consulta.";
        }
        iniciarSesionUsuario(req, resp, u, saludo);
```

```
    }

    private void registrarNuevoCliente(HttpServletRequest req,
HttpServletResponse resp)
            throws ServletException, IOException {
        String nombreCompleto = req.getParameter("nombreCompleto");
        String nuevoUsuario = req.getParameter("nuevoUsuario");
        String nuevaClave = req.getParameter("nuevaClave");

        String error = usuarios.registrarCliente(nombreCompleto, nuevoUsuario,
nuevaClave);
        if (error != null && !error.isEmpty()) {
            req.setAttribute("error", error);
            req.setAttribute("mostrarRegistroCliente", true);
            req.setAttribute("regNombre", nombreCompleto);
            req.setAttribute("regUsuario", nuevoUsuario);
            req.setAttribute("roles", UsuarioService.ROLES);
            req.setAttribute("bdConectada", ConexionBD.estaDisponible());
            req.setAttribute("bdEstado", ConexionBD.getEstadoTexto());
            Controlador.vista(req, resp, "login");
            return;
        }
        Usuario u = usuarios.autenticarUsuario(nuevoUsuario, nuevaClave,
Usuario.ROL_CLIENTE, req.getRemoteAddr());
        iniciarSesionUsuario(req, resp, u,
                "Cuenta de Cliente creada correctamente. ¡Bienvenido a NEXORA, "
+ u.getNombreCompleto() + "!");
    }

    private void iniciarSesionUsuario(HttpServletRequest req, HttpServletResponse
resp, Usuario u, String mensajeBienvenida)
            throws IOException {
        // Preservar urlRetorno (ej. comprar producto) antes de invalidar la
sesión
        HttpSession anterior = req.getSession(false);
        String urlRetorno = null;
        if (anterior != null) {
            Object ret = anterior.getAttribute("urlRetorno");
            if (ret instanceof String s && !s.isBlank()) {
                urlRetorno = s;
            }
            anterior.invalidate();
        }
        HttpSession sesion = req.getSession(true);
        sesion.setAttribute("usuario", u.getUsername());
        sesion.setAttribute("nombreCompleto", u.getNombreCompleto());
        sesion.setAttribute("rol", u.getRol());
        sesion.setAttribute("usuarioObj", u);
        sesion.setAttribute("modoBD", ConexionBD.getEstadoTexto());
        Controlador.mensaje(req, "ok", mensajeBienvenida);
        if (urlRetorno != null && !urlRetorno.isBlank()) {
            resp.sendRedirect(urlRetorno);
            return;
```

```
        }
        resp.sendRedirect(req.getContextPath() + "/inicio");
    }
}
```